Guía Integrante 1: carga, limpieza y transformación

Nov 2, 2026 · @Jose

La parte 1 del pipeline está terminada y funciona de forma modular: descarga los viajes de taxi amarillo de Nueva York de enero de 2024, los limpia, crea las variables nuevas y deja una tabla de **2.801.330 viajes y 20 columnas** en Parquet, lista para el EDA y el modelo.

Esta guía es un documento interno del equipo: sirve para entender el código y para estudiar. No reemplaza al informe PDF de la entrega, que se redacta aparte.

Resumen

Etapa	Filas	Cambio
Dataset original	2.964.624	—
Después de la limpieza	2.811.704	−152.920 (5,2 %)
Después del filtro de viajes absurdos	2.801.330	−10.374 (0,4 %)
Tabla final	**2.801.330**	Se conserva el 94,5 %

- **Fuente:** registros de viajes de la TLC de Nueva York (TLC Trip Record Data), archivo `yellow_tripdata_2024-01.parquet`, más la tabla de zonas `taxi_zone_lookup.csv`.
- **Objetivo acordado para el modelo:** predecir `fare_amount` (tarifa del taxímetro, en USD) antes del viaje. Es una regresión.
- **Salida:** carpeta `data/processed/taxi_limpio` (Parquet, 4 archivos).

Cómo ejecutarlo

Todo se corre desde la **raíz del proyecto**, con el entorno virtual activo:

```
python -m pip install -r requirements.txt
python main.py
```

La primera vez descarga el dataset (unos 50 MB); las siguientes detecta que ya existe y no lo vuelve a bajar. Al terminar imprime las filas finales y deja el Parquet en `data/processed/taxi_limpio`.

Requisitos:

- Python 3.12 o superior y Java 17 o superior (se usó Temurin 21).
- `pandas<3`: PySpark todavía no soporta del todo pandas 3.
- **Solo en Windows:** para escribir el Parquet, Spark necesita `winutils.exe` y `hadoop.dll` en `C:\hadoop\bin`, la variable `HADOOP_HOME=C:\hadoop` y `C:\hadoop\bin` en el `Path`. En Colab no hace falta.

La carpeta `data/` no está en GitHub (está en `.gitignore`). Cada uno la genera corriendo `main.py`, que es también lo que va a hacer el docente.

Para leer la tabla desde el EDA o el modelo:

```
from src import config
df = spark.read.parquet(config.RUTA_PROCESSED)
```

Se le pasa la carpeta, no un archivo: Spark junta las cuatro partes solo.

Flujo del pipeline

`main.py` crea una sola sesión de Spark y llama a cada módulo en este orden. Cada función recibe un DataFrame y devuelve otro, así los pasos se encadenan.

1. **Sesión** (`sesion_spark.crear_sesion`): crea la SparkSession local que usan todos los módulos.
2. **Carga** (`carga_datos.cargar_datos` y `cargar_zonas`): descarga los dos archivos con `requests` si no existen, los lee con Spark y valida que estén las columnas esperadas.
3. **Limpieza** (`limpieza.limpiar_dataset`): corrige tipos, imputa nulos con la moda, saca duplicados y filtra valores imposibles.
4. **Transformación** (`transformacion.transformar_datos`): crea las variables nuevas, une las zonas para obtener los distritos, filtra viajes con duración o velocidad absurdas y deja las 20 columnas finales.
5. **Guardado** (`transformacion.guardar_procesado`): escribe el Parquet en `data/processed/taxi_limpio`.
6. **Siguiente paso del equipo:** el EDA (Integrante 2) y el modelo (Integrante 3) leen ese Parquet.

Las constantes que usan todos los módulos (URLs, rutas, listas de columnas, umbrales de los filtros) están en `config.py`. Si hay que cambiar un umbral o una ruta, se cambia ahí y en ningún otro lado.

Módulos y funciones

Las funciones que terminan coordinando un módulo (`cargar_datos` , `limpiar_dataset` , `transformar_datos`) son las únicas que llama `main` . Las demás son piezas internas.

`config.py`

Solo constantes, sin funciones. Las rutas se arman desde la raíz del proyecto con `Path(__file__)` , para que funcionen se ejecute desde donde se ejecute.

Constante	Para qué
`NOMBRE_APP`	Nombre de la sesión de Spark
`DATASET_URL` , `ZONAS_URL`	De dónde se descargan los dos archivos
`RUTA_TAXI_RAW` , `RUTA_ZONAS_RAW` , `RUTA_PROCESSED`	Dónde se guardan los archivos
`COLUMNAS_REQUERIDAS`	Columnas que el archivo original tiene que traer
`DICT_TIPOS`	Columnas a convertir y su tipo destino
`COLUMNAS_IMPUTAR`	Columnas cuyos nulos se completan con la moda
`FECHA_INICIO` , `FECHA_FIN` , `DISTANCIA_MAX_MILLAS`	Umbrales del filtro de limpieza
`ZONAS_AEROPUERTO`	IDs de zona de Newark (1), JFK (132) y LaGuardia (138)
`DURACION_MIN` , `DURACION_MAX` , `VELOCIDAD_MAX`	Umbrales del filtro de viajes absurdos
`COLUMNAS_FINALES`	Las 20 columnas de la tabla de salida

`sesion_spark.py`

Función	Qué hace
`crear_sesion(nombre_app)`	Crea la SparkSession en modo local con todos los núcleos, oculta las barras de progreso y baja el nivel de log a ERROR

`carga_datos.py`

Función	Qué hace
`descargar_dataset(url, ruta)`	Descarga cualquier archivo por partes con `requests`; si ya existe, no lo baja de nuevo. Se usa para el Parquet y para el CSV
`leer_dataset(spark, ruta)`	Lee el Parquet de viajes
`leer_zonas(spark, ruta)`	Lee el CSV de zonas con `header` e `inferSchema`
`validar_columnas(df, columnas)`	Comprueba que estén todas las columnas; si falta alguna, corta con un error que dice cuál
`cargar_datos(spark)`	Coordina descarga, lectura y validación de los viajes
`cargar_zonas(spark)`	Coordina descarga y lectura de las zonas

`limpieza.py`

Función	Qué hace
`corregir_tipos(df, tipos)`	Convierte `passenger_count`, `RatecodeID` y `payment_type` de bigint a int
`tratar_nulos(df, columnas)`	Marca las filas con nulos en `nulos_imputados` y los completa con la moda
`eliminar_duplicados(df)`	Saca filas idénticas
`filtrar_invalidos(df)`	Saca montos, distancias, pasajeros, códigos y fechas imposibles
`limpiar_dataset(df)`	Coordina todo, muestra filas antes y después, y guarda el resultado en memoria (`cache`)

`transformacion.py`

Función	Qué hace
`agregar_variables_tiempo(df)`	`hora`, `dia_semana`, `es_fin_de_semana`
`agregar_variables_velocidad(df)`	`duracion_min` y `velocidad_mph` (protegida contra la división por cero)

Función	Qué hace
`agregar_es_aeropuerto(df, zonas)`	`es_aeropuerto` (0/1)
`agregar_distritos(df, zonas)`	`distrito_subida` y `distrito_bajada`, con dos broadcast joins
`filtrar_viajes_absurdos(df)`	Saca duraciones fuera de 1 a 180 minutos y velocidades fuera de 0 a 80 mph
`seleccionar_columnas(df, columnas)`	Deja las 20 columnas finales
`guardar_procesado(df, ruta)`	Escribe el Parquet en 4 archivos (`coalesce(4)`)
`transformar_datos(df, zonas)`	Coordina todo lo anterior en orden

Decisiones de limpieza

La regla general fue: **limpieza saca lo imposible, el EDA analiza lo raro.** Una tarifa de USD 5.000 es extrema pero posible, así que queda para el EDA; una tarifa negativa es imposible y se elimina.

Problema detectado	Evidencia (`describe()`)	Decisión	Por qué
Tipos	3 columnas de códigos en bigint	Pasarlas a int	Consistencia y menos memoria; Parquet ya traía el resto bien tipado
Fechas	`timestamp_ntz`	No tocarlas	Conserva la hora local de Nueva York; convertirlas podría correrlas a la hora argentina
Nulos	140.162 filas (4,7 %) con nulos a la vez en `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge` y `Airport_fee`	Imputar `passenger_count` y `RatecodeID` con la moda (1 en ambas) y marcar las filas en `nulos_imputados`	Son conteos y códigos: el promedio no tiene sentido. La marca permite probar el modelo con y sin esas filas

Problema detectado	Evidencia (`describe()`)	Decisión	Por qué
Nulos en recargos	Mismas filas	No tratarlos	`congestion_surcharge` y `Airport_fee` se descartan en la transformación
Montos	`fare_amount` mínimo −899, `total_amount` −900	Quedarse con montos > 0	Son reembolsos o anulaciones, no viajes
Distancia	Mínimo 0, máximo 312.722 millas	Entre 0 y 100 millas (excluye 0)	Un viaje sin distancia o de cientos de millas es un error de registro
Pasajeros	Mínimo 0, máximo 9	Entre 1 y 6	Un viaje sin pasajeros no tiene sentido
Tipo de tarifa	Máximo 99	`RatecodeID` entre 1 y 6	99 significa desconocido
Fechas fuera del mes	Viajes registrados fuera de enero de 2024	Subida desde el 1/1/2024 y antes del 1/2/2024	El archivo es de enero
Bajada antes de la subida	—	Bajada posterior a la subida	Es imposible
Duplicados	—	Eliminar filas idénticas	Inflan conteos y sesgan el modelo

Filtro de viajes absurdos (en transformación): una vez calculadas `duracion_min` y `velocidad_mph`, se dejaron solo los viajes de 1 a 180 minutos y de 0 a 80 mph. Va en transformación porque esas columnas no existen antes. Eliminó 10.374 viajes.

Para el informe falta el detalle de cuántas filas saca cada condición por separado. Se calcula una sola vez con `df.filter(<condición inversa>).count()` en un archivo de prueba, no dentro del pipeline.

Tabla final

El Parquet tiene 20 columnas: 11 originales y 9 creadas. **Num** = cantidad que se puede sumar o promediar; **Cat** = código o categoría, aunque sea un número.

Columna	Origen	Tipo	Qué es	Uso
`tpep_pickup_datetime`	Original	Fecha	Fecha y hora de subida	EDA (series de tiempo)
`trip_distance`	Original	Num	Distancia en millas	Modelo y EDA
`RatecodeID`	Original	Cat	1 normal, 2 JFK, 3 Newark, 4 Nassau/Westchester, 5 negociada, 6 grupal	Modelo y EDA
`PULocationID`	Original	Cat	Zona de subida (1 a 265)	Modelo y EDA
`DOLocationID`	Original	Cat	Zona de bajada (1 a 265)	Modelo y EDA
`passenger_count`	Original	Num	Pasajeros (1 a 6)	EDA
`payment_type`	Original	Cat	0 flex, 1 tarjeta, 2 efectivo, 3 sin cargo, 4 disputa	Solo EDA
`fare_amount`	Original	Num	Tarifa del taxímetro, USD	**Objetivo del modelo**
`tip_amount`	Original	Num	Propina, USD (solo se registra con tarjeta)	Solo EDA
`tolls_amount`	Original	Num	Peajes, USD	Solo EDA
`total_amount`	Original	Num	Total cobrado, USD	Solo EDA
`duracion_min`	Nueva	Num	Bajada menos subida, en minutos	Solo EDA
`velocidad_mph`	Nueva	Num	Distancia sobre duración, en millas por hora	Solo EDA
`hora`	Nueva	Num	Hora de subida (0 a 23)	Modelo y EDA
`dia_semana`	Nueva	Cat	**1 = domingo, 7 = sábado**	Modelo y EDA
`es_fin_de_semana`	Nueva	Cat (0/1)	1 si es sábado o domingo	Modelo y EDA

Columna	Origen	Tipo	Qué es	Uso
`es_aeropuerto`	Nueva	Cat (0/1)	1 si sube o baja en un aeropuerto, o la tarifa es de aeropuerto	Modelo y EDA
`distrito_subida`	Nueva (join)	Cat	Manhattan, Brooklyn, Queens, Bronx, Staten Island, EWR, Unknown, N/A	EDA y dashboard
`distrito_bajada`	Nueva (join)	Cat	Ídem, para la bajada	EDA y dashboard
`nulos_imputados`	Nueva	Booleano	Verdadero si la fila tenía nulos que se completaron con la moda	Control de calidad

Los códigos de `RatecodeID` y `payment_type` salen del diccionario de datos de la TLC; conviene verificarlos ahí antes de citarlos en el informe.

Para los Integrantes 2 y 3

Integrante 2 (EDA y SQL)

- **Los cálculos van en Spark** (es el enfoque "EDA a gran escala con Spark"): `groupBy`, `agg`, `summary`, correlaciones. Sin AutoEDA.
- **Los gráficos van en Plotly** sobre tablas ya agregadas, pasadas a pandas con `toPandas()`. Nunca `toPandas()` sobre la tabla completa.
- Para gráficos de puntos sueltos (por ejemplo, distancia vs. tarifa), usar una muestra de 10 a 50 mil filas: `df.sample(fraction=0.01, seed=config.SEMILLA)`.
- **El EDA final se corre con las 2,8 millones de filas**; la muestra es solo para desarrollar más rápido.
- Cuidado con `dia_semana`: 1 es domingo, no lunes. Para ordenar o rotular gráficos hay que tenerlo en cuenta.
- Las propinas en efectivo no están en el dataset: cualquier análisis de propinas va solo con `payment_type = 1` (tarjeta).
- Los viajes de JFK tienen tarifa fija: en ese grupo la distancia casi no influye en el precio. Es un buen hallazgo para mostrar.
- **El cierre del EDA es la lista de variables para el modelo**, con un número que justifique cada una. Esa lista es la integración entre los dos enfoques que pide la consigna.

Integrante 3 (modelo y dashboard)

- **Objetivo:** `fare_amount`.
- **Entradas candidatas:** `trip_distance`, `RatecodeID`, `PULocationID`, `DOLocationID`, `hora`, `dia_semana`, `es_fin_de_semana`, `es_aeropuerto` (la lista final la define el EDA).
- **No usar como entradas** (filtración de datos, se conocen recién al cobrar): `total_amount`, `tip_amount`, `tolls_amount`, `payment_type`, `duracion_min`, `velocidad_mph`.
- `RatecodeID`, las zonas y `dia_semana` son **categorías**, no cantidades: en MLlib conviene tratarlas con `StringIndexer` (y `OneHotEncoder` para modelos lineales).
- Con Random Forest, las zonas tienen 265 valores: hay que subir `maxBins` a 265 o más, o usar los distritos en su lugar.
- `nulos_imputados` sirve para comparar el modelo con y sin las filas imputadas.
- Usar `config.SEMILLA` en el `randomSplit` para que los resultados sean reproducibles.

Problemas encontrados y soluciones

Si les aparece alguno de estos errores, ya sabemos qué es.

Error o síntoma	Causa	Solución
`no se encontró Python`	Python no está en el PATH (o se desactivó Anaconda)	Usar `py -m venv .venv`, o instalar Python tildando "Add python.exe to PATH"
`Access Denied` al abrir la URL	Se usó el patrón `AAAA-MM` literal	Poner la fecha real: `2024-01`
`'NoneType' object has no attribute ...`	Una función sin `return`, o se asignó el resultado de una función que no devuelve nada	Revisar el `return` de la función de la línea anterior
`UNRESOLVED_COLUMN ... cannot be resolved`	La columna ya se había sacado con un `select` anterior	Seleccionar columnas solo al final de la transformación
`ConnectionRefusedError` o `dagScheduler() is null`	Se ejecutó `spark.stop()` y después se siguió usando la sesión en el notebook	Reiniciar el kernel; en notebooks, `spark.stop()` solo en la última celda

Error o síntoma	Causa	Solución
`name '__file__' is not defined`	Se pegó `config.py` en una celda del notebook	Importar `config` en vez de copiarlo
`HADOOP_HOME and hadoop.home.dir are unset` al guardar	Windows necesita `winutils.exe` para escribir	Instalar winutils y configurar `HADOOP_HOME` (ver Cómo ejecutarlo)
`ImportError: cannot import name ...`	Se importa la función desde el módulo equivocado, o el archivo no se guardó	Revisar en qué `.py` está definida y guardar con Ctrl + S
Aparece "Welcome to Spark" al correr `main.py`	Algún módulo tiene `from pyspark.shell import spark` (lo agrega el autocompletado de VS Code)	Buscar `pyspark.shell` en el proyecto y borrar esa línea
Los archivos aparecen en `src/data/`	Rutas relativas (`./data`) ejecutadas desde `src/`	Armar las rutas desde la raíz con `Path(__file__)` en `config.py`
`IndentationError`	Mezcla de tabulaciones y espacios	`Ctrl + Shift + P` → "Convert Indentation to Spaces"

Dos reglas que evitan la mayoría de estos problemas:

- En el notebook, importar los módulos de `src/` y no copiar funciones en celdas. Después de cambiar un `.py`, reiniciar el kernel (o usar `%autoreload 2`).
- Correr siempre desde la raíz del proyecto.

Guía de estudio: conceptos de Spark usados

Estos son los conceptos que aparecen en el código de esta parte. Cada uno indica dónde se usa, para poder señalarlo en el video o en el coloquio.

Concepto	Qué es	Dónde se usa
SparkSession	La puerta de entrada a Spark. Vive en el Driver, el proceso que arma el plan y coordina a los executors	`crear_sesion`; se crea una sola vez en `main` y se pasa como parámetro

Concepto	Qué es	Dónde se usa
`getOrCreate()`	Crea la sesión o reutiliza la que ya existe, para no tener dos sesiones a la vez	`crear_sesion`
`local[*]`	Spark corre en la propia computadora y usa todos los núcleos como executors	`crear_sesion`
DataFrame	Tabla distribuida: los datos están divididos en particiones repartidas entre executors	Todo el pipeline
Inmutabilidad	Ninguna operación modifica un DataFrame: siempre devuelve uno nuevo. Por eso cada línea es `df = df...`	Todas las funciones
Lazy evaluation	Las transformaciones solo arman un plan; Spark ejecuta recién con una acción	Encadenar `withColumn` y `filter` es instantáneo; el trabajo ocurre en `count()` o al escribir
Transformación vs. acción	Transformaciones: `filter`, `select`, `withColumn`, `join`. Acciones: `count`, `show`, `first`, `write`	`limpiar_dataset` hace solo dos `count` para no recorrer los datos de más
Parquet	Formato columnar que guarda el esquema (nombres y tipos) dentro del archivo	Lectura sin `inferSchema`; salida del pipeline
`cache()`	Guarda un DataFrame en memoria la primera vez que se calcula, para no recalcularlo	Al final de `limpiar_dataset`
Broadcast join	La tabla chica se copia a cada executor y el join se hace en el lugar, sin mover la tabla grande por la red (sin *shuffle*)	`agregar_distritos`: 265 zonas contra 2,8 millones de viajes
Shuffle	Redistribuir filas entre executors por la red; es la operación más cara de Spark	Lo que evita el broadcast join
`coalesce(n)`	Reduce la cantidad de particiones juntando las existentes, sin shuffle	`guardar_procesado`: 4 archivos en vez de unos 30

Concepto	Qué es	Dónde se usa
Escritura distribuida	Cada partición se escribe como un archivo `part-` en paralelo; `_SUCCESS` indica que terminó bien	Carpeta `taxi_limpio`
`F.when`	Un if por fila; sin `otherwise` deja nulo	`velocidad_mph`, para evitar la división por cero
Modo ANSI	En Spark 4, operaciones inválidas como dividir por cero pueden tirar error en vez de devolver nulo	Motivo de la protección en `velocidad_mph`

Conceptos de Python que también conviene repasar: diferencia entre parámetro y argumento, `if __name__ == "__main__":`, por qué los imports van arriba del archivo, list comprehensions (el conteo de nulos), y `return` frente a `None`.

Para repasar comandos, el Machete PySpark tiene todos los usados con ejemplos.

Preguntas probables del coloquio

Respuestas cortas para practicar en voz alta. En el coloquio pueden preguntarle a cualquiera sobre cualquier módulo, así que los tres deberían poder responderlas.

1. **¿Por qué Spark y no pandas?** Son casi 3 millones de filas. Spark reparte el trabajo en paralelo entre núcleos (y escalaría a un clúster), mientras que pandas carga todo en la memoria de una sola máquina.
2. **¿Por qué se descarga con `requests` y no con Spark?** `spark.read` lee archivos de un sistema de archivos (disco, HDFS, S3), no URLs HTTP. Spark está hecho para procesar datos, no para transferirlos.
3. **¿Qué es la lazy evaluation?** Las transformaciones no se ejecutan en el momento: Spark arma un plan y lo ejecuta todo junto cuando hay una acción, como `count()` o escribir un archivo. Así puede optimizar el plan completo.
4. **¿Por qué cada línea es `df = df...`?** Los DataFrames son inmutables: cada operación devuelve uno nuevo. Sin asignarlo, el cambio se pierde.
5. **¿Por qué la sesión se recibe como parámetro en vez de crearla en cada módulo?** Debe haber un solo Driver coordinando. Se crea una vez en `main` y se pasa a quien la necesite; `getOrCreate` además evita duplicarla.

6. **¿Por qué se imputaron los nulos con la moda y no con el promedio?** `RatecodeID` es un código: el promedio (2,07) no corresponde a ninguna tarifa. `passenger_count` es un conteo entero. Para categorías y conteos, la moda es lo que corresponde.
7. **¿Por qué no se usaron las filas con nulos tal como estaban?** Los nulos estaban juntos en cinco columnas de las mismas 140.162 filas, un patrón sistemático. Se imputaron y se marcaron en `nulos_imputados` para poder evaluar su efecto.
8. **¿Qué diferencia hay entre limpiar y filtrar outliers?** La limpieza saca valores imposibles (tarifas negativas, distancia 0). Los valores extremos pero posibles los analiza el EDA.
9. **¿Qué es un broadcast join y por qué se usó?** La tabla de zonas tiene 265 filas. Spark manda una copia a cada executor, que hace el join con sus viajes en el lugar, y se evita el shuffle de los 2,8 millones de viajes.
10. **¿Por qué left join?** Para no perder ningún viaje aunque su zona no figure en la tabla; en ese caso el distrito queda nulo.
11. **¿Por qué `es_aeropuerto` usa las zonas y no `Airport_fee`?** Ese recargo solo se cobra al subir en el aeropuerto, así que no detecta los viajes hacia el aeropuerto, y es nulo en las filas imputadas.
12. **¿Por qué la velocidad usa `F.when`?** Para no dividir por cero cuando la duración es 0. En Spark 4 eso puede tirar error; con `when` queda nulo y el filtro de viajes absurdos saca la fila.
13. **¿Por qué la salida es una carpeta con varios archivos?** Cada partición se escribe en paralelo como un archivo `part-`. Se lee pasando la carpeta. `coalesce(4)` junta las particiones para no tener decenas de archivos chicos.
14. **¿Por qué Parquet y no CSV?** Es columnar (lee solo las columnas que se usan), está comprimido y guarda el esquema, así que los tipos no se pierden ni hay que inferirlos.
15. **¿Qué es la filtración de datos y cómo la evitaron?** Es usar como entrada información que no estaría disponible al momento de predecir. El modelo estima la tarifa antes del viaje, así que no usa el total, la propina, los peajes ni la duración.

Pendientes antes de la entrega

- [ ] Pasar todas las rutas de `config.py` a `RAIZ` (hoy siguen siendo `./data/...`) y borrar la carpeta `src/data`.
- [ ] Volver a poner en `COLUMNAS_REQUERIDAS` las columnas originales del dataset (hoy valida las 11 finales).
- [ ] Unificar `RUTA_PROCESSED` / `RUTA_PROCESADO` en un solo nombre.
- [ ] Sacar el `df.show(5)` de `main.py` y ordenar los mensajes de consola por etapa.
- [ ] Buscar `pyspark.shell` en todo el proyecto y borrar esa línea donde aparezca.

- [ ] Completar legajo, Entrada y Salida en el encabezado de cada módulo.
- [ ] Cambiar `pandas` por `pandas<3` en `requirements.txt` y fijar las versiones con `pip freeze`.
- [ ] Agregar al README la nota de `winutils` para Windows.
- [ ] Calcular cuántas filas saca cada filtro por separado, para el informe.
- [ ] Probar el pipeline completo en Colab, como lo va a correr el docente.
- [ ] Revisar en GitHub que `src/config.py` esté subido.